

nginx ca reverse proxy

Exerciții · pe `compose-platform.yaml`, fișierul tău

Lucrezi pe fișierul tău real, cel din `lectii/L1-containerizare/rezolvare/compose-platform.yaml`, cu cele cinci aplicații și MySQL comun. Nu pe unul pregătit dinainte.

Pașii ghidați acoperă **două** aplicații: `school-app` și `sebi-online-shop-app`. Celelalte trei rămân pentru provocări — le adaugi singur, după ce vezi tiparul.

Fiecare pas are un criteriu de gata. Nu treci mai departe până nu-l vezi cu ochii tăi în terminal. „Pornește” nu e criteriu.

Pasul 1 — Închide MySQL

În `compose-platform.yaml`, serviciul `mysql` are:

```
ports:
  - "3307:3306"
```

Șterge ambele linii. Complet — nu le muta pe `127.0.0.1`, aici nu e nevoie nici de atât. Aplicațiile ajung la bază pe rețeaua internă Docker, după numele serviciului (`jdbcm:mysql://mysql:3306/...`), lucru pe care îl faci deja.

```
docker compose down
docker compose up -d
```

CRITERIU DE GATA

```
docker compose ps                # mysql: coloana PORTS goala
docker compose logs school-app | tail -20 # fara eroare de conexiune
```

Dacă aplicațiile merg mai departe fără portul publicat, ai dovedit singur că nu-l folosea nimeni din afară.

Pasul 2 — Leagă cele două aplicații pe loopback

La `school-app` și `sebi-online-shop-app`, adaugă `127.0.0.1`: în față și pune context path-ul:

```
school-app:
  image: sebi123q/school-app-api:0.0.1
  ports:
    - "127.0.0.1:8094:8080"
  environment:
    SERVER_PORT: "8080"
    SERVER_SERVLET_CONTEXT_PATH: /school
    ...

sebi-online-shop-app:
  image: sebi123q/sebi-online-shop-app:1.0
  ports:
    - "127.0.0.1:8093:8089"
  environment:
    SERVER_PORT: "8089"
    SERVER_SERVLET_CONTEXT_PATH: /shop
    ...
```

Nu reconstruiești nimic: `SERVER_SERVLET_CONTEXT_PATH` se citește din mediu, exact ca `SERVER_PORT` pe care îl folosești deja.

ATENȚIE — adresa s-a mutat

De la pasul ăsta, Swagger **nu** mai e la `:8094/swagger-ui/...`, ci la `:8094/school/swagger-ui/...`. Dacă primești 404, nu e stricat — e context path-ul care își face treaba.

CRITERIU DE GATA

```
curl -I http://127.0.0.1:8094/school/swagger-ui/index.html
curl -I http://127.0.0.1:8093/shop/swagger-ui/index.html
```

Amândouă întorc `200`. Rulezi comenzile **pe mașina unde stau containerele** — `127.0.0.1` înseamnă mereu „mașina de pe care tizez”, deci de pe laptop n-ai cum să verifici serverul așa.

Pasul 3 — Instalează nginx pe server

```
ssh -i <cheia>.pem ubuntu@<IP_PUBLIC>
```

```
sudo apt update
sudo apt install -y nginx
sudo systemctl status nginx
```

Nu în Docker — ca pachet pe server, la fel cum ai instalat Docker.

CRITERIU DE GATA

`systemctl status` arată `active (running)`, iar în browser `http://<IP_PUBLIC>/` afișează „Welcome to nginx!”.

Ca să vezi pagina, portul 80 trebuie deschis în Security Group. Dacă browserul se învârtă fără să încarce, ăla e motivul — nu nginx.

Pasul 4 — Scie configurația

Pe server, creează `~/platform.conf`. Un singur bloc `server`, cu trei `location` -uri:

```
server {
    listen 80 default_server;
    server_name _;

    location /school {
        proxy_pass http://127.0.0.1:8094;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }

    location /shop {
        proxy_pass http://127.0.0.1:8093;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }

    location / {
        default_type text/plain;
        return 200 "platform up\n";
    }
}
```

CRITERIU DE GATA

Cele două porturi din `proxy_pass` sunt exact cele pe care le-ai pus la pasul 2 în `ports:` — verifică-le una lângă alta, e greșeala numărul unu.

Validarea cu `nginx -t` vine la pasul următor: `nginx` nu poate testa un fișier care stă în `~`, îl citește doar din `/etc/nginx/`.

Pasul 5 — Activează și pornește

```
cd ~/platform
docker compose up -d
docker compose ps

curl http://127.0.0.1:8094/school/swagger-ui/index.html

sudo mv ~/platform.conf /etc/nginx/sites-available/platform
sudo ln -sf /etc/nginx/sites-available/platform \
        /etc/nginx/sites-enabled/platform
sudo rm /etc/nginx/sites-enabled/default

sudo nginx -t && sudo systemctl reload nginx
```

Ordinea contează: întâi containerele răspund pe loopback, abia apoi pui portarul în fața lor. Altfel debughezi `nginx` pentru o problemă care e în aplicație.

CRITERIU DE GATA

`nginx -t` spune `syntax is ok / test is successful`, iar din browser, de pe laptop:

```
http://<IP_PUBLIC>/                                -> "platform up"
http://<IP_PUBLIC>/school/swagger-ui/index.html
http://<IP_PUBLIC>/shop/swagger-ui/index.html
```

Toate trei pe portul 80, fără număr de port în adresă.

Pasul 6 — Închide firewall-ul

În AWS Console → EC2 → Security Groups → grupul instanței tale, lasă exact două reguli inbound:

Type	Port	Source
SSH	22	My IP
HTTP	80	Anywhere (0.0.0.0/0)

Șterge tot ce e pe `8091 – 8095` și `3307`.

CRITERIU DE GATA — de pe laptop

```
curl -sS -m 5 http://<IP_PUBLIC>/school/swagger-ui/index.html
curl -sS -m 5 http://<IP_PUBLIC>:8093
curl -sS -m 5 http://<IP_PUBLIC>:3307
```

Prima **trebuie** să întoarcă HTML. Ultimele două **trebuie** să dea eroare (refused sau timeout). Dacă toate trei răspund, lecția nu e făcută — e doar instalată.

Când ceva nu merge

Ce vezi	Unde te uiți întâi
502 Bad Gateway	nginx merge, containerul nu răspunde. Pe server: <code>curl 127.0.0.1:<port>/...</code> și <code>docker compose ps</code>
404 pe Swagger	context path. Prefixul din <code>location</code> trebuie să fie identic cu <code>SERVER_SERVLET_CONTEXT_PATH</code>
„Welcome to nginx!” pe /	n-ai șters <code>sites-enabled/default</code>
Browserul se învâрте	Security Group, nu nginx. Timeout = firewall; refused = nu ascultă nimeni
Config bun, dar nu se aplică	ai uitat <code>reload</code> după ce ai editat

Cele două comenzi de log:

```
sudo tail -f /var/log/nginx/error.log
docker compose logs -f school-app
```

Provocări

1. Mai ai trei aplicații în compose: `boats-users`, `solarsyncbroker-app` și `sebi-school-app`. Adaugă-le în spatele aceleiași porți, pe rute proprii. Serverul trebuie să rămână cu **un singur port public**.

2. În `proiecte/online-shop-cloud-spring-api/docker-compose-online-shop.yaml`, linia `DB_URL` se termină cu un caracter în plus, care n-are ce căuta acolo. Găsește-l **fără** să pornești containerul, apoi verifică prin rulare că aveai dreptate.

Întrebarea grea: de ce `docker compose config` **nu** îl prinde, deși prinde alte greșeli din același fișier?

3. Oprește instanța EC2 (Stop), apoi pornește-o din nou (Start). Ce se strică și de ce? Ce ai schimba în configurație ca să nu se mai strice?